

1. Fundamentos del Diseño de Software

¿Qué es el Diseño de Software?

- Etapa donde se analizan requerimientos y se obtiene una descripción de la estructura interna del software
- Sirve como base para la construcción del software

Arquitectura vs. Diseño

- Arquitectura: Define estructura general, contexto de solución, componentes
- Diseño: Especifica detalles de implementación de cada componente

Importancia del Diseño

- Análisis y evaluación de modelos
- Búsqueda de alternativas de solución
- Negociación de requerimientos
- Insumo para implementación y pruebas

Factores que Influyen en la Arquitectura

- Producto: Requerimientos funcionales y de calidad
- Restricciones: Tecnológicas, organizacionales, legales

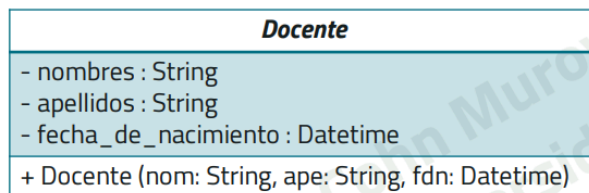
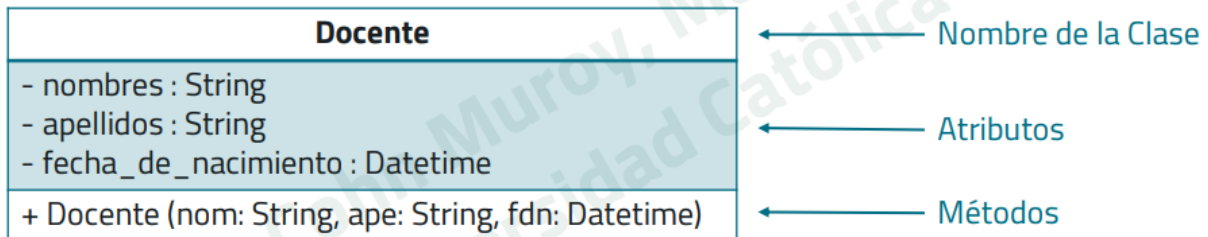
Modelo C4 para Vistas Arquitectónicas

- Contexto: Sistema como caja negra
 - Contenedores: Módulos autocontenidos
 - Componentes: Bloques de construcción internos
 - Código: Diagramas detallados (clases, secuencia, ER)
-

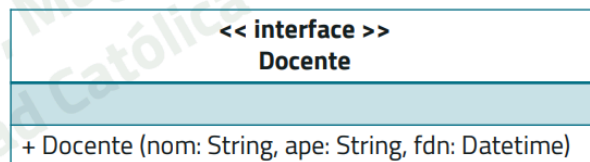
2. Notación UML - Clases de Diseño

Objetos vs. Clases

- Objeto: Entidad con identidad única (física o conceptual)
- Clase: Descripción que agrupa objetos con propiedades comunes



Clase Abstracta



Interfaz

NOTACIÓN DE ATRIBUTOS

Visibilidad	Nombre del Atributo	:	Tipo de Dato	= Valor por defecto (opcional)
público (+) privado (-) protegido (#) paquete (~)	<u>Nombre del Atributo</u> (estático)		string int double etc.	

Ejemplos

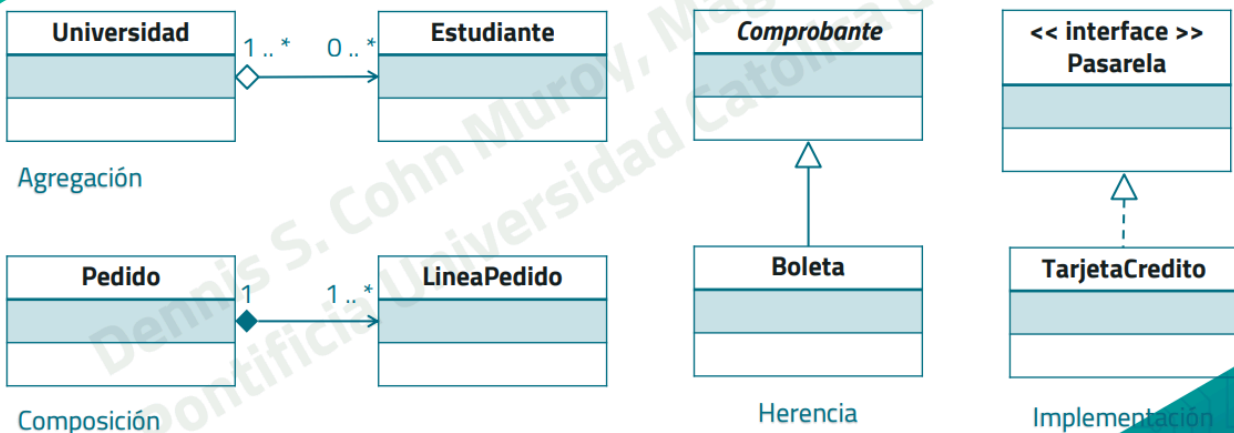
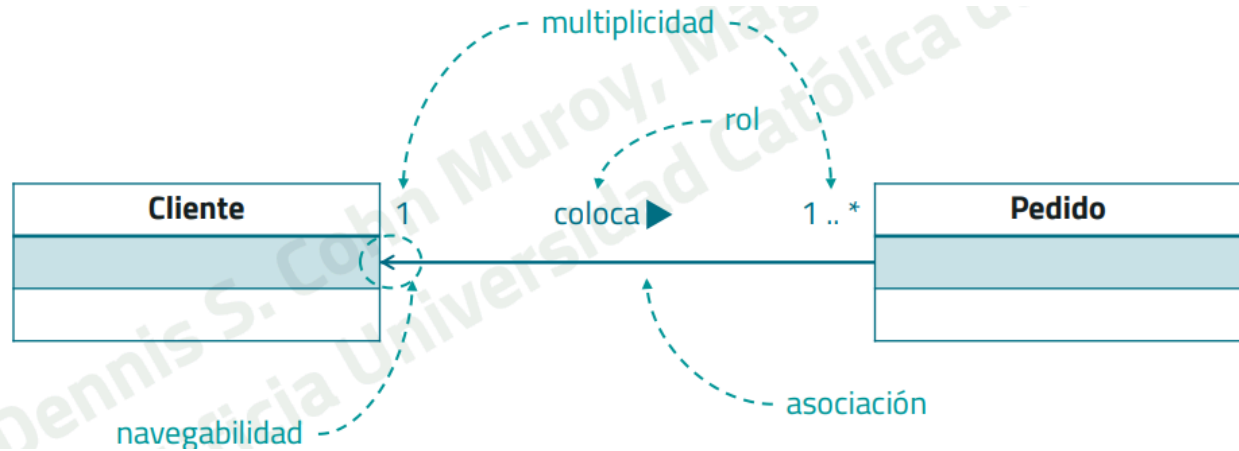
- | | | |
|---------------------------|---|--|
| # identificador : int = 0 | → | atributo protegido de tipo int con nombre "identificador" y valor inicial "0". |
| - nombres : string | → | atributo privado de tipo string con nombre "nombres". |

NOTACIÓN DE MÉTODOS

Visibilidad	Nombre del Método	(Parámetros)	:	Tipo de dato de salida
público (+) privado (-) protegido (#) paquete (~)	<u>Nombre del Método</u> (estático)	Nombre : tipo de dato		string, int, double, etc. (Omitir en el constructor) (void es opcional)

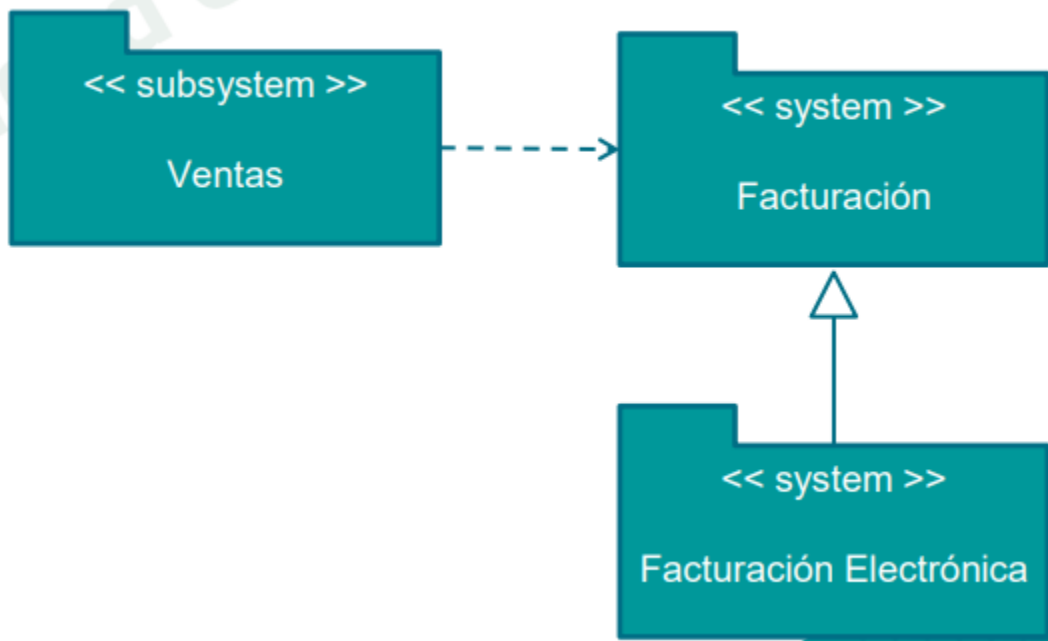
Ejemplos

+ iniciarSesión () → método público para iniciar sesión en el sistema.
 + obtenerInstancia () : ConectorBD → método estático público que retorna una instancia del ConectorBD.
 - validarDatos (nombres : string, apellidos: string) : boolean → método privado que valida los nombres y apellidos; retorna un valor lógico.



Relaciones entre Clases

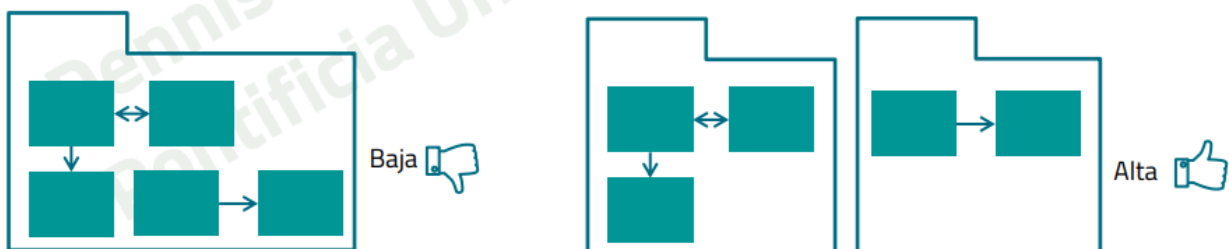
- Asociación: Conexión estructural
- Agregación: "Tiene-un" (relación débil)
- Composición: "Parte-de" (relación fuerte)
- Herencia: Generalización-especialización
- Implementación: Clase → Interfaz



Acoplamiento: Dependencias entre componentes.



Cohesión: Relación de los elementos dentro de un componente.



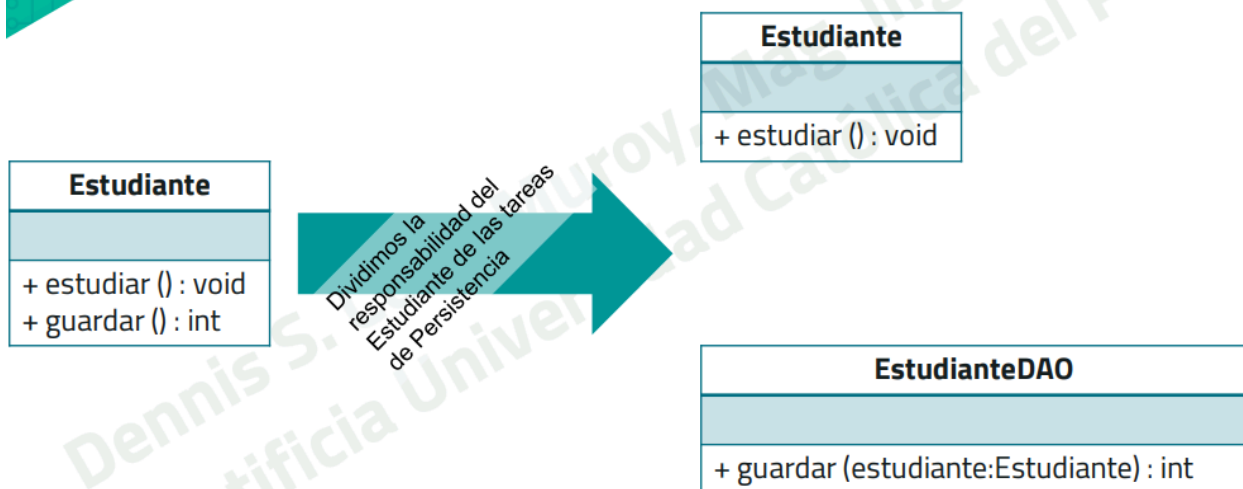
Principios SOLID

- Single Responsibility: Una clase, una responsabilidad
- Open/Closed: Abierto extensión, cerrado modificación
- Liskov Substitution: Subtipos reemplazables
- Interface Segregation: Interfaces específicas
- Dependency Inversion: Dependier de abstracciones

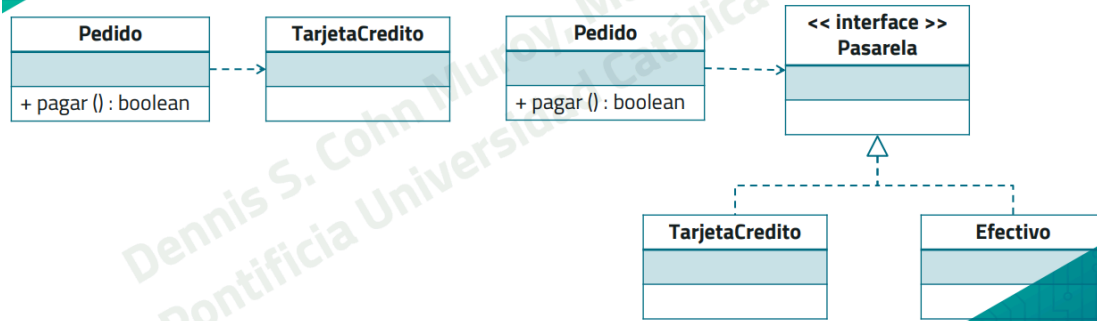
Otros Principios

- KISS: Keep It Small & Simple
- DRY: Don't Repeat Yourself
- Modularidad: Bajo acoplamiento, alta cohesión

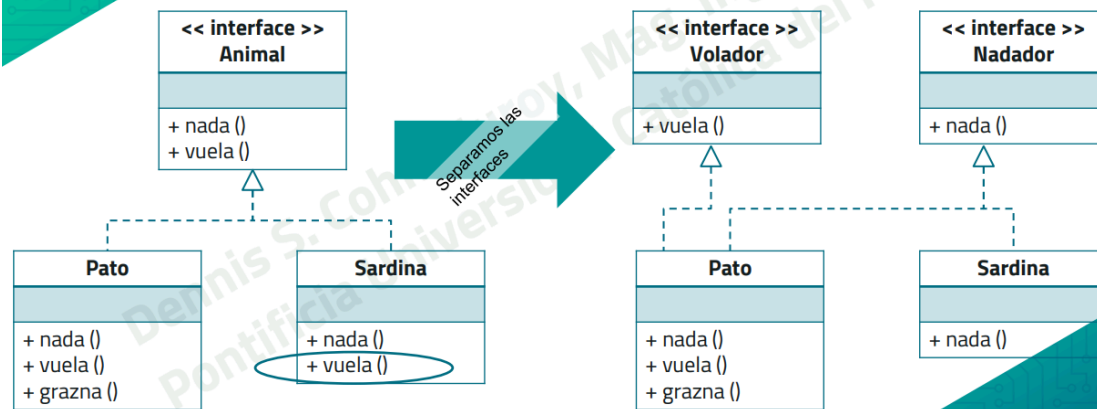
S.O.L.I.D: SINGLE RESPONSIBILITY



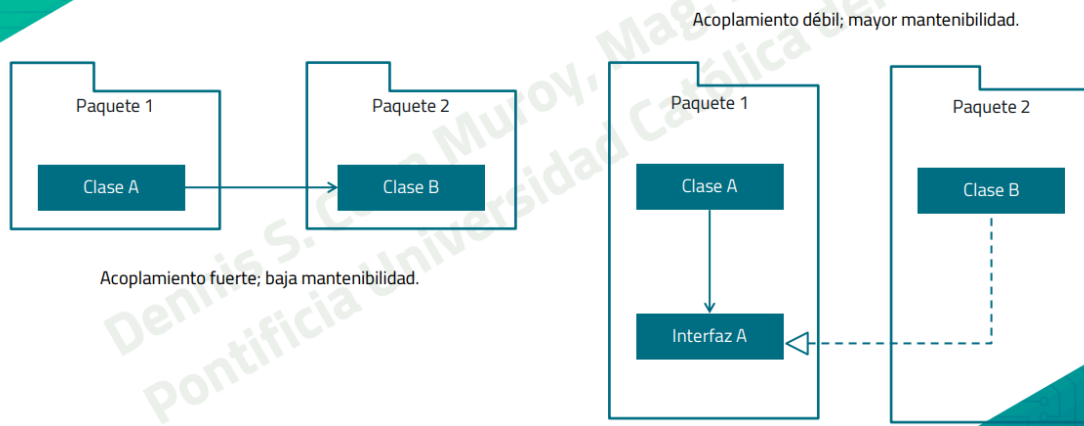
S.O.L.I.D: OPEN / CLOSED



S.O.L.I.D: INTERFACE SEGREGATION



S.O.L.I.D: DEPENDENCY INVERSION



3. Patrones de Diseño

¿Qué son los Patrones?

- Soluciones probadas a problemas recurrentes de diseño
- "Microarquitecturas" que optimizan tiempo de desarrollo

Elementos de un Patrón

- Nombre: Identificador
- Problema: Cuándo aplicarlo
- Solución: Diseño, relaciones y responsabilidades
- Consecuencias: Resultados e implicancias

Clasificación de Patrones

Por Alcance

- Clase: Relaciones de herencia (estáticas)
- Objeto: Relaciones entre objetos (dinámicas)

Por Propósito

- Creacionales: Creación de objetos (Factory, Singleton, Builder)
- Estructurales: Organización de estructuras (Adapter, Decorator, Facade)
- Comportamiento: Algoritmos y responsabilidades (Observer, Strategy, Command)

Tipos de Patrones

- Análisis: Problemas de conceptualización
- Arquitectura: Estructuras de subsistemas completos
- Diseño: Grupos pequeños de objetos interrelacionados
- Codificación: Específicos a lenguajes de programación